... which we've done with a mix of `XPath::query()` and `HTMLDocument::querySelector()`. Note the `::query()[0]` syntax is equivalent to `::querySelector()` – we're grabbing the first matched element in the returned arrays, so we're working with `Dom\Node` instances (or some sub-class). Next we'll use `Node::insertBefore()` to put a "new" node in front of an existing – this method is smart enough to realize when the "new" node is actually an existing node in the DOM, in which case the end result is the node repositioned relative to the child...

```
// $lastPara is inserted before $firstPara
$main->insertBefore($lastPara, $firstPara);
```

... after this call is made, the `$lastParam` node will have been moved to the first-child position, and `$firstPara` will now be the second child of `main`, which we can prove by writing out the new markup using `HTMLDocument::saveHTML($node)`:

```
print $dom->saveHTML($main);
```

```
Output:
<main class="main-content">
    <p class="small">Para Three</p><p class="large">Para One</p>
    <p class="small">Para Two</p>
</main>
```

Don't worry about the lack of newline character between the closing and opening tags of the first `p` children of `main`; it's still a perfectly well-formed HTML fragment. Speaking of which, what we have generated here isn't a full document, which should be obvious from the lack of a `<!DOCTYPE` declaration and missing `head`, `title`, and `body` elements. `HTMLDocument::saveHTML()` takes a DOM node object argument and will generate the HTML from that point of the tree. Handy if you just need to generate a fragment for use elsewhere. If you want to save the entire document, call it without arguments:

```
print $dom->saveHTML();
Output:
<!DOCTYPE html><html><head>
  <meta charset="utf-8"></meta>
  <title>HTML5 is lovely</title>
  ...
```